

Part 1

- Task 1:

See the script `get_interleaving.sh`.

- Task 2: Pattern Analysis

In the 1,000 runs, `f1` finished first 995 times, while `f2` finished first 5 times. Therefore, `f1` generally finishes before `f2` in this experiment. This is likely because `fprintf()` is a buffered library function, whereas each call to `write()` enters the kernel and has system-call overhead. However, `fprintf()` still periodically flushes its buffer, so it is not entirely performed only in memory.

- Task 3: Consistency Analysis

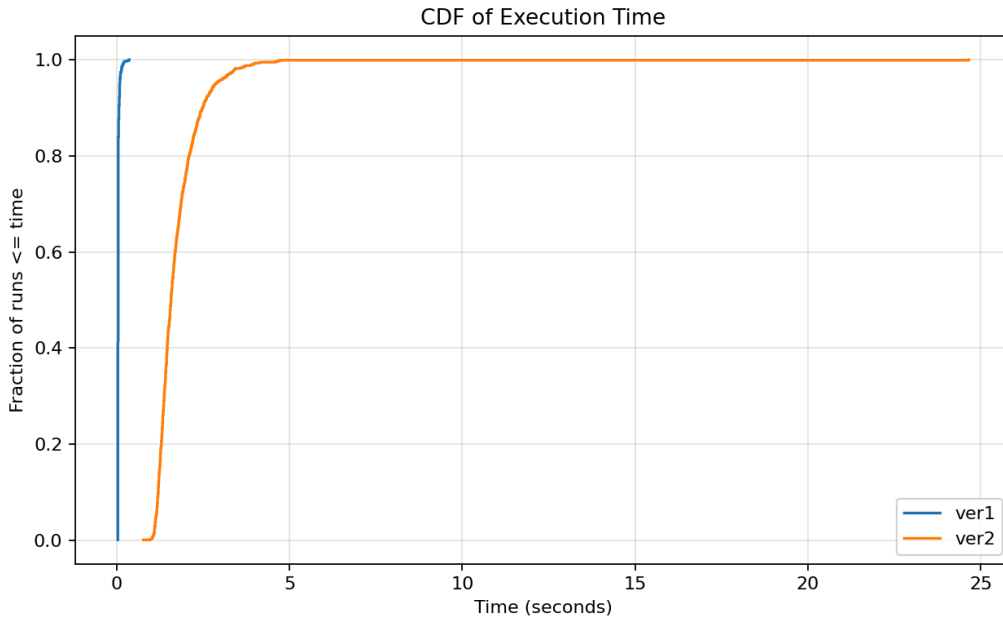
This ordering is not guaranteed across all runs. Although `f1` usually finishes first because of buffering and the lower number of system calls, thread execution depends on factors such as CPU scheduling, system load, cache state, and disk activity. Consequently, `f2` may occasionally finish first, even though this is expected to happen rarely.

Part 2

- Task 1:

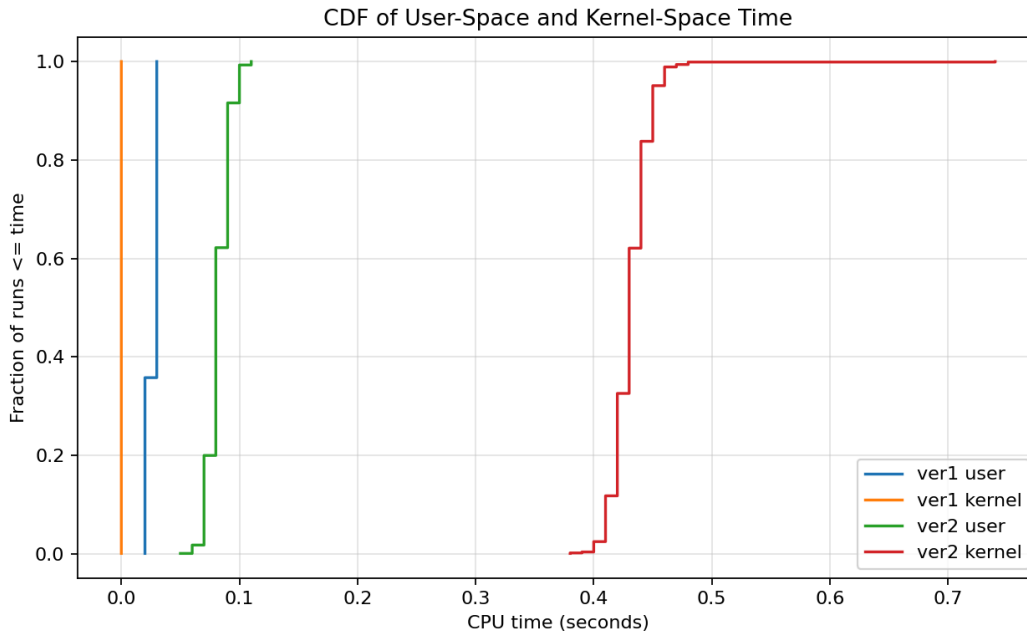
See the script `measure_times.sh`.

- Task 2: Execution-time CDF



From the figure `execution_time_cdf.png`, we can observe that `ver1` completes nearly all runs below 0.1 s, while `ver2` is centered around roughly 1.5 s and has a long tail reaching about 24.7 s. Therefore, `ver2` is both slower and less stable across runs. This occurs because `ver1` uses buffered `fprintf()` calls, while `ver2` performs 100,000 individual `write()` system calls. Each `write()` requires a transition into the kernel, creating significant overhead.

- Task 3: User-space and kernel-space CDF



From the figure `user_kernel_time_cdf.png`, we can observe that the kernel-space Cdf shows the largest difference between the programs. `ver2` spends approximately 0.43 s of CPU time in the kernel because every `write()` call enters the kernel. `ver1` uses buffered `fprintf()` calls, so most writes are handled in userspace buffers and are flushed less frequently. As a result, `ver1` spends substantially less kernel time.

- Task 4:

Based on measurements, the execution times were not the same across runs:

- `ver1`: median 0.04 s, maximum 0.36 s.
- `ver2`: median 1.57 s, maximum 24.66 s.

`ver1` had relatively stable execution times, while `ver2` showed much greater variation, likely because it performs many system calls and it's more affected by scheduling and I/O delays.

Execution times can be made more stable by:

- Running the experiment on an idle system.

- Pinning the process to a specific CPU core.
- Keeping the input file and executable in the file system cache.
- Avoiding other disk-intensive or CPU-intensive processes.
- Increasing the process priority with `nice` or `chrt`.